Relational Algebra ↔ SQL Dictionary

Database Systems Quick Reference

Complete Translation Guide

Fall 2025

Contents

1 Quick Translation Table **2**

2 Basic Operations **2**
- 2.1 SELECT (Filter Rows) . . . 2
- 2.2 PROJECT (Select Columns) . . . 3
- 2.3 Combined: Filter and Project . . . 4

3 Join Operations **4**
- 3.1 Cartesian Product . . . 4
- 3.2 Natural Join . . . 5
- 3.3 Theta Join (Conditional Join) . . . 6
- 3.4 JOIN with USING . . . 6
- 3.5 Outer Joins . . . 6

4 Set Operations **7**
- 4.1 UNION . . . 7
- 4.2 INTERSECTION . . . 7
- 4.3 DIFFERENCE (EXCEPT) . . . 8

5 Aggregate Functions and Grouping **9**

6 Nested Subqueries **10**

7 Complete Query Examples **10**
- 7.1 Example 1: Multi-Table Join . . . 10
- 7.2 Example 2: Filter + Join + Project . . . 11
- 7.3 Example 3: Set Difference . . . 12
- 7.4 Example 4: Aggregation with Filter . . . 12

8 Common Patterns Quick Reference **13**

9 Key Differences Summary **14**

10 Operator Correspondence Table **14**

11 Tips for Translation **15**

12 Practice Exercises **16**

1 Quick Translation Table

At-A-Glance Reference

Operation	Relational Algebra	SQL
Filter rows	$\sigma_{\text{condition}}(R)$	`WHERE condition`
Select columns	$\Pi_{\text{A,B}}(R)$	`SELECT A, B`
All columns	-	`SELECT *`
Rename	$\rho_S(R)$	`AS S`
Cartesian Product	$R \times S$	`FROM R, S`
Natural Join	$R \bowtie S$	`NATURAL JOIN`
Theta Join	$R \bowtie_\theta S$	`JOIN ... ON`
Union	$R \cup S$	`UNION`
Intersection	$R \cap S$	`INTERSECT`
Difference	$R - S$	`EXCEPT`
Aggregation	$\gamma_{;F}(R)$	`SELECT AGG(...)`
Grouping	$\gamma_{L;F}(R)$	`GROUP BY L`
Remove duplicates	(automatic)	`DISTINCT`
Keep duplicates	-	`ALL`

2 Basic Operations

2.1 SELECT (Filter Rows)

Relational Algebra vs SQL: Filtering

Relational Algebra:

$$\sigma_{\text{condition}}(R)$$

Notation: σ (sigma)
Purpose: Select rows that satisfy condition
Example:

$$\sigma_{\text{Age}>21}(\text{Student})$$

$$\sigma_{\text{Dept='CS'}\wedge\text{Salary}>70000}(\text{Instructor})$$

SQL:

```
WHERE condition
```

Keyword: WHERE
Purpose: Filter rows based on condition
Example:

```
SELECT *
FROM Student
WHERE Age > 21;

SELECT *
FROM Instructor
WHERE Dept = 'CS'
  AND Salary > 70000;
```

Complete Side-by-Side Example

Task: Find instructors in CS department with salary > 70000

Relational Algebra	SQL

$\sigma_{\text{dept_name='CS'}\wedge\text{salary}>70000}(\text{instructor})$	`SELECT *`
	`FROM instructor`
	`WHERE dept_name = 'CS'`
	`  AND salary > 70000;`

Given Table: instructor

name	dept_name	salary
Srinivasan	CS	65000
Katz	CS	75000
Brandt	CS	92000
Wu	Finance	90000

Result:

name	dept_name	salary
Katz	CS	75000
Brandt	CS	92000

2.2 PROJECT (Select Columns)

Relational Algebra vs SQL: Column Selection

Relational Algebra:

$$\Pi_{A_1,A_2,\ldots}(R)$$

Notation: Π (pi)
Purpose: Select specific columns
Removes duplicates automatically
Example:

$$\Pi_{\text{name, salary}}(\text{instructor})$$

$$\Pi_{\text{dept_name}}(\text{instructor})$$

SQL:

```
SELECT col1, col2, ...
```

Keyword: SELECT
Purpose: Specify which columns to retrieve
Keeps duplicates by default
Example:

```
SELECT name, salary
FROM instructor;

SELECT DISTINCT dept_name
FROM instructor;
```

Key Difference: Duplicates!

Relational Algebra: Automatically removes duplicates (relations are sets)
SQL: Keeps duplicates by default (use `DISTINCT` to remove)

Relational Algebra	SQL
$\Pi_{\text{dept_name}}(\text{instructor})$	`SELECT DISTINCT dept_name FROM instructor;`
(no duplicate option)	`SELECT dept_name FROM instructor;` (keeps duplicates)

PROJECT Example with Duplicates

Task: Get all department names
Given Table: instructor

name	dept_name
Srinivasan	CS
Katz	CS
Wu	Finance
Mozart	Music

Relational Algebra	SQL (with DISTINCT)

$\Pi_{\text{dept_name}}(\text{instructor})$
Result:
dept_name: CS, Finance, Music
	`SELECT DISTINCT dept_name`
	`FROM instructor;`
	Result:
	dept_name: CS, Finance, Music

SQL without DISTINCT:

```
SELECT dept_name FROM instructor;  -- Returns: CS, CS, Finance, Music
```

2.3 Combined: Filter and Project

Combining Operations

Pattern: Filter rows, then select columns

Relational Algebra	SQL

$\Pi_{\text{name}}(\sigma_{\text{dept_name='CS'}}(\text{instructor}))$
Order: Inside-out
1. σ filters rows
2. Π selects columns
	`SELECT name`
	`FROM instructor`
	`WHERE dept_name = 'CS';`
	Order: Sequential
	1. FROM: source table
	2. WHERE: filter rows
	3. SELECT: choose columns

Filter + Project Example

Task: Get names of CS instructors with salary > 70000

Relational Algebra	SQL

$\Pi_{\text{name}}(\sigma_{\text{dept_name='CS'}\wedge\text{salary}>70000}(\text{instructor}))$	`SELECT name`
	`FROM instructor`
	`WHERE dept_name = 'CS'`
	`  AND salary > 70000;`

3 Join Operations

3.1 Cartesian Product

Cartesian Product

Relational Algebra:

$$R \times S$$

Symbol: $\times$
Result: All possible pairs
Example:

$$\text{Student} \times \text{Course}$$

SQL:

```
FROM R, S
```

Keyword: Comma in FROM
Result: All possible combinations
Example:

```
SELECT *
FROM Student, Course;
```

3.2 Natural Join

Natural Join: Automatic Matching

Relational Algebra:

$$R \bowtie S$$

Symbol: $\bowtie$
Behavior:

- Joins on all common attributes
- Equality condition
- One copy of common attrs

Example:

$$\text{Student} \bowtie \text{Enrollment}$$

SQL:

```
NATURAL JOIN
```

Keyword: NATURAL JOIN
Behavior:

- Joins on all common columns
- Equality condition
- One copy of common cols

Example:

```
SELECT *
FROM Student
NATURAL JOIN Enrollment;
```

Natural Join Example

Tables:
Student:

ID	Name
1	Alice
2	Bob

Enrollment:

ID	Course
1	CS101
2	CS102

Relational Algebra	SQL

$\text{Student} \bowtie \text{Enrollment}$	`SELECT *`
	`FROM Student`
	`NATURAL JOIN Enrollment;`

Result:

ID	Name	Course
1	Alice	CS101
2	Bob	CS102

Note: ID appears only once!

3.3 Theta Join (Conditional Join)

Theta Join: Custom Conditions

Relational Algebra:

$$R \bowtie_\theta S$$

where θ is a condition
Equivalent to:

$$\sigma_\theta(R \times S)$$

Example:

$$\text{Student} \bowtie_{\text{Student.ID=Enrollment.ID}} \text{Enrollment}$$

SQL:

```
JOIN ... ON condition
```

Keywords: JOIN ... ON
Equivalent to:

```
FROM R, S
WHERE condition
```

Example:

```
SELECT *
FROM Student
JOIN Enrollment
ON Student.ID = Enrollment.ID;
```

Three Ways to Write the Same Join

Task: Join Student and Enrollment on ID

Method	Query
Relational Algebra	$\text{Student} \bowtie_{\text{Student.ID=Enrollment.ID}} \text{Enrollment}$
SQL: JOIN ON	`SELECT * FROM Student JOIN Enrollment ON Student.ID = Enrollment.ID;`
SQL: WHERE	`SELECT * FROM Student, Enrollment WHERE Student.ID = Enrollment.ID;`
SQL: NATURAL	`SELECT * FROM Student NATURAL JOIN Enrollment;`

All produce the same result!

3.4 JOIN with USING

JOIN USING: Safer Alternative

Relational Algebra:
Natural join equivalent, but explicit about which attributes

$$\text{Student} \bowtie \text{Enrollment}$$

(on ID only)

SQL:

```
JOIN ... USING (column)
```

Example:

```
SELECT *
FROM Student
JOIN Enrollment
USING (ID);
```

3.5 Outer Joins

Outer Joins: Keeping Unmatched Rows

Relational Algebra:

- Left: $R \bowtie S$
- Right: $R \bowtie S$
- Full: $R \bowtie S$

Behavior: Keep unmatched rows, fill with NULL

SQL:

```
LEFT OUTER JOIN
RIGHT OUTER JOIN
FULL OUTER JOIN
```

Example:

```
SELECT *
FROM Student
LEFT OUTER JOIN Enrollment
ON Student.ID = Enrollment.ID;
```

4 Set Operations

4.1 UNION

Union: Combine Results

Relational Algebra:

$$R \cup S$$

Symbol: $\cup$
Behavior:

- All rows from R or S
- Removes duplicates
- Requires union-compatibility

Example:

$$\text{CS_Students} \cup \text{Math_Students}$$

SQL:

```
UNION
```

Behavior:

- All rows from both queries
- Removes duplicates
- Requires compatible schemas

Example:

```
SELECT * FROM CS_Students
UNION
SELECT * FROM Math_Students;
```

UNION ALL: Keep Duplicates

SQL only: Use `UNION ALL` to keep duplicates

```
SELECT * FROM CS_Students
UNION ALL
SELECT * FROM Math_Students;
```

Relational Algebra: No direct equivalent (relations are sets, no duplicates)

4.2 INTERSECTION

Intersection: Common Rows

Relational Algebra:

$$R \cap S$$

Symbol: $\cap$
Behavior: Rows in both R and S
Example:

$$\text{CS_Students} \cap \text{Math_Students}$$

(students in both departments)

SQL:

```
INTERSECT
```

Behavior: Rows in both queries
Example:

```
SELECT * FROM CS_Students
INTERSECT
SELECT * FROM Math_Students;
```

(students in both departments)

4.3 DIFFERENCE (EXCEPT)

Difference: Exclusion

Relational Algebra:

$$R - S$$

Symbol: $-$ or $\backslash$
Behavior: Rows in R but not in S
NOT commutative!
Example:

$$\text{CS_Students} - \text{Math_Students}$$

(only CS, not both)

SQL:

```
EXCEPT
```

Behavior: Rows in first query but not second
NOT commutative!
Example:

```
SELECT * FROM CS_Students
EXCEPT
SELECT * FROM Math_Students;
```

(only CS, not both)

Set Operations Complete Example

Given Tables:
Fall2017:

course_id
CS101
CS347
PHY101

Spring2018:

course_id
CS101
CS315
FIN201

Operation 1: UNION (Courses in Fall OR Spring)

Relational Algebra:

$$\text{Fall2017} \cup \text{Spring2018}$$

Result: CS101, CS315, CS347, FIN201, PHY101

SQL:

```
SELECT * FROM Fall2017
UNION
SELECT * FROM Spring2018;
```

Operation 2: INTERSECT (Courses in Fall AND Spring)

Relational Algebra:

$$\text{Fall2017} \cap \text{Spring2018}$$

Result: CS101

SQL:

```
SELECT * FROM Fall2017
INTERSECT
SELECT * FROM Spring2018;
```

Operation 3: EXCEPT (Courses in Fall but NOT Spring)

Relational Algebra:

$$\text{Fall2017} - \text{Spring2018}$$

Result: CS347, PHY101

SQL:

```
SELECT * FROM Fall2017
EXCEPT
SELECT * FROM Spring2018;
```

5 Aggregate Functions and Grouping

Aggregation in RA vs SQL

Relational Algebra:
Notation: $\gamma_{L;F}(R)$

- **L:** List of grouping attributes
- **F:** Aggregate function(s)
- **R:** Relation

Aggregate without grouping:

$$\gamma_{;\text{avg(salary)}}(\text{instructor})$$

Aggregate with grouping:

$$\gamma_{\text{dept_name;avg(salary)}}(\text{instructor})$$

Multiple aggregates:

$$\gamma_{\text{dept_name;count}(*),\text{avg(salary)}}(\text{instructor})$$

SQL:
Built-in aggregate functions:

- `AVG(column)`
- `COUNT(*)`
- `COUNT(column)`
- `SUM(column)`
- `MIN(column)`
- `MAX(column)`

Example:

```
SELECT AVG(salary)
FROM instructor;

SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

Aggregation Examples

Task: Find average salary per department

Relational Algebra	SQL

$\gamma_{\text{dept_name;avg(salary)}}(\text{instructor})$	`SELECT dept_name,`
	`  AVG(salary)`
	`FROM instructor`
	`GROUP BY dept_name;`

Given Table: instructor

dept_name	salary
CS	75000
CS	65000
Finance	90000
Finance	80000

Result:

dept_name	AVG(salary)
CS	70000
Finance	85000

6 Nested Subqueries

Subqueries: Set Membership

Relational Algebra:
Use set operations and joins
Example: Students in CS101

$\Pi_{\text{name}}(\text{Student} \bowtie (\sigma_{\text{course_id='CS101'}}(\text{Enrollment})))$

SQL:
Use `IN` with subquery
Example:

```
SELECT name
FROM Student
WHERE ID IN
  (SELECT ID
   FROM Enrollment
   WHERE course_id = 'CS101');
```

Subquery Operators

SQL provides special operators:

- `IN` / `NOT IN`: Set membership
- `EXISTS` / `NOT EXISTS`: Test if result non-empty
- `SOME` / `ALL`: Comparison with set
- `UNIQUE`: Test for duplicates

Relational Algebra: Express using combinations of basic operations

7 Complete Query Examples

7.1 Example 1: Multi-Table Join

Students with Course Titles

Task: Get student names with their course titles
Schema:

- Student(<u>SID</u>, Name)
- Enrollment(<u>SID, CID</u>)
- Course(<u>CID</u>, Title)

BEFORE:
Student:

SID	Name
1	Alice
2	Bob
3	Carol

Enrollment:

SID	CID
1	101
2	102
1	102

Course:

CID	Title
101	Databases
102	Algorithms

Relational Algebra	SQL

$\Pi_{\text{Name, Title}}((\text{Student} \bowtie \text{Enrollment}) \bowtie \text{Course})$
 	
Steps:
1. Join Student & Enrollment
2. Join result with Course
3. Project Name, Title
	`SELECT Name, Title`
	`FROM Student`
	`NATURAL JOIN Enrollment`
	`NATURAL JOIN Course;`
	
	or
	
	`SELECT S.Name, C.Title`
	`FROM Student S,`
	`  Enrollment E,`
	`  Course C`
	`WHERE S.SID = E.SID`
	`  AND E.CID = C.CID;`

AFTER:

Name	Title
Alice	Databases
Bob	Algorithms
Alice	Algorithms

7.2 Example 2: Filter + Join + Project

CS Students with Courses

Task: Find names and course titles for CS students
Schema: Student(SID, Name, Major), Enrollment(SID, CID), Course(CID, Title)
BEFORE:

Student:

SID	Name	Major
1	Alice	CS
2	Bob	Math
3	Carol	CS

Enrollment:

SID	CID
1	101
2	102
3	101

Course:

CID	Title
101	Databases
102	Algorithms

Relational Algebra:

$$\Pi_{\text{Name, Title}}(\sigma_{\text{Major='CS'}}(\text{Student}) \bowtie \text{Enrollment}) \bowtie \text{Course})$$

SQL:

```
SELECT S.Name, C.Title
FROM Student S,
     Enrollment E,
     Course C
WHERE S.Major = 'CS'
  AND S.SID = E.SID
  AND E.CID = C.CID;
```

AFTER: (Only CS students)

Name	Title
Alice	Databases
Carol	Databases

7.3 Example 3: Set Difference

Students NOT Enrolled

Task: Find students who haven't enrolled in any course
Schema: Student(SID, Name), Enrollment(SID, CID)
BEFORE:
Student:

SID	Name
1	Alice
2	Bob
3	Carol
4	Dave

Enrollment:

SID	CID
1	101
2	102
1	102

Relational Algebra:

$$\Pi_{\text{SID}}(\text{Student}) - \Pi_{\text{SID}}(\text{Enrollment})$$

Then join with Student for names:

$$\text{Student} \bowtie (\Pi_{\text{SID}}(\text{Student}) - \Pi_{\text{SID}}(\text{Enrollment}))$$

SQL:

Method 1: EXCEPT

```
SELECT SID FROM Student
EXCEPT
SELECT SID FROM Enrollment;
```

Method 2: NOT IN

```
SELECT Name
FROM Student
WHERE SID NOT IN
  (SELECT SID
   FROM Enrollment);
```

AFTER: (Students not in Enrollment)

SID	Name
3	Carol
4	Dave

7.4 Example 4: Aggregation with Filter

Departments with High Average Salary

Task: Find departments where average salary > 70000
Schema: instructor(ID, name, dept_name, salary)
BEFORE:

instructor:

ID	name	dept_name	salary
1	Einstein	Physics	95000
2	Mozart	Music	40000
3	Tesla	Physics	87000
4	Turing	Comp. Sci.	75000
5	Beethoven	Music	56000
6	Dijkstra	Comp. Sci.	92000

Relational Algebra:

$$\sigma_{\text{avg_sal}>70000}(\gamma_{\text{dept_name};\text{avg(salary)}\to\text{avg_sal}}(\text{instructor}))$$

SQL:

```
SELECT dept_name,
       AVG(salary) AS avg_sal
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 70000;
```

AFTER: (Departments with avg salary > 70000)

dept_name	avg_sal
Physics	91000
Comp. Sci.	83500

Key Point: Use `HAVING` (not `WHERE`) to filter after aggregation!

8 Common Patterns Quick Reference

Pattern Translation Guide

1. Basic Operations:

Pattern	RA	SQL
Filter rows	$\sigma_{\text{cond}}(R)$	`...WHERE cond`
Select columns	$\Pi_{A,B}(R)$	`SELECT A, B...`
Filter & select	$\Pi_A(\sigma_{\text{cond}}(R))$	`SELECT A...WHERE cond`

2. Join Operations:

Pattern	RA	SQL
Natural join	$R \bowtie S$	`...NATURAL JOIN...`
Conditional join	$R \bowtie_{R.x=S.y} S$	`...JOIN...ON R.x=S.y`
Multiple joins	$(R \bowtie S) \bowtie T$	`R JOIN S JOIN T`

3. Set Operations:

Pattern	RA	SQL
Union	$R \cup S$	`(...) UNION (...)`
Intersection	$R \cap S$	`(...) INTERSECT (...)`
Difference	$R - S$	`(...) EXCEPT (...)`

4. Aggregation:

Pattern	RA	SQL
Count rows	$\gamma_{;\text{count}(*)}(R)$	`SELECT COUNT(*)`
Group & aggregate	$\gamma_{A;\text{agg}(B)}(R)$	`...GROUP BY A`
Filter groups	$\sigma_{\text{cond}}(\gamma_{A;\ldots}(R))$	`...HAVING cond`

9 Key Differences Summary

Critical Differences to Remember

1. **Duplicates:**
   - RA: Always removes (relations are sets)
   - SQL: Keeps by default (use `DISTINCT` to remove)
2. **Execution Order:**
   - RA: Inside-out (innermost first)
   - SQL: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
3. **Notation:**
   - RA: Mathematical symbols ($\sigma, \Pi, \bowtie$, etc.)
   - SQL: English keywords (`WHERE, SELECT, JOIN`, etc.)
4. **Filtering Aggregates:**
   - RA: Apply σ after aggregation
   - SQL: Use `HAVING` (not `WHERE`)
5. **Subqueries:**
   - RA: Use combinations of basic operations
   - SQL: Direct support for nested queries (`IN, EXISTS`, etc.)
6. **NULL Handling:**
   - RA: Typically ignores NULL (theoretical)
   - SQL: Three-valued logic (TRUE, FALSE, UNKNOWN)

10 Operator Correspondence Table

Complete Operator Reference

Operation	RA Symbol	SQL Keyword	Notes
Select (filter)	σ	WHERE	Horizontal filtering
Project (columns)	Π	SELECT	Vertical filtering
Rename relation	ρ	AS	Alias for table
Rename attribute	-	AS	Alias for column
Cartesian product	$\times$	FROM R, S	All combinations
Natural join	$\bowtie$	NATURAL JOIN	Automatic equality
Theta join	$\bowtie_\theta$	JOIN ... ON	Custom condition
Join using	-	JOIN ... USING	Specific columns
Left outer join	$⟕$	LEFT OUTER JOIN	Keep left unmatched
Right outer join	$⟖$	RIGHT OUTER JOIN	Keep right unmatched
Full outer join	$⟗$	FULL OUTER JOIN	Keep all unmatched
Union	$\cup$	UNION	Remove duplicates
Union (keep dups)	-	UNION ALL	Keep duplicates
Intersection	$\cap$	INTERSECT	Common rows
Difference	$-$ or $\setminus$	EXCEPT	Exclusion
Assignment	$\leftarrow$	CREATE VIEW	Store result
Aggregation (no group)	$\gamma_{;F}$	Aggregate funcs	COUNT, SUM, etc.
Grouping & aggregation	$\gamma_{L;F}$	GROUP BY + AGG	Partition and aggregate
Filter groups	σ after γ	HAVING	Post-aggregation filter
Remove duplicates	(automatic)	DISTINCT	Explicit in SQL
Ordering	-	ORDER BY	Sort results
Limit results	-	LIMIT / TOP	First N rows

11 Tips for Translation

RA to SQL Translation Strategy

Step-by-step approach:

1. **Identify the operations:** What RA operations are used?
2. **Find base relations:** What tables are mentioned?
3. **Start with FROM:** List all base tables
4. **Add joins:** Translate $\bowtie$ to JOIN or WHERE conditions
5. **Add filters:** Translate σ to WHERE
6. **Add grouping:** Translate γ to GROUP BY
7. **Add column selection:** Translate Π to SELECT
8. **Add set operations:** Translate $\cup, \cap, -$ to UNION, INTERSECT, EXCEPT

SQL to RA Translation Strategy

Step-by-step approach:

1. **Start with FROM:** Identify base relations
2. **Handle joins:** Convert JOIN to $\bowtie$ or $\times$ with σ
3. **Convert WHERE:** Map to σ
4. **Convert GROUP BY:** Map to $\gamma_{\text{attrs;agg}}$
5. **Convert HAVING:** Apply σ after aggregation

6. **Convert SELECT:** Map to Π
7. **Handle set ops:** Convert UNION/INTERSECT/EXCEPT to $\cup / \cap / -$
8. **Build inside-out:** Innermost operations first

12 Practice Exercises

Translation Practice

Translate between RA and SQL:

1. RA to SQL:

$$\Pi_{\text{name}}(\sigma_{\text{salary}>80000}(\text{instructor}))$$

Answer:

```
SELECT name FROM instructor WHERE salary > 80000;
```

2. SQL to RA:

```
SELECT title FROM course WHERE credits = 3;
```

Answer:

$$\Pi_{\text{title}}(\sigma_{\text{credits}=3}(\text{course}))$$

3. RA to SQL (with join):

$$\Pi_{\text{name, course_id}}(\text{instructor} \bowtie \text{teaches})$$

Answer:

```
SELECT name, course_id
FROM instructor NATURAL JOIN teaches;
```

4. SQL to RA (with aggregation):

```
SELECT dept_name, COUNT(*)
FROM instructor
GROUP BY dept_name;
```

Answer:

$$\gamma_{\text{dept_name;count}(*)}(\text{instructor})$$